

App Store Publishing Checklist

ARTickle Academy • Google Play + Apple App Store

Current state. ARTickle Academy is a **React + Vite web app** hosted on **Firebase Hosting** (Firestore + Cloud Functions backend). There is currently **no native iOS or Android project**. The website on its own cannot be submitted to either store — it must first be **wrapped in a native shell**. This document is the end-to-end checklist to get there.

0. Pick a packaging approach (do this first)

Route	Android	iOS	Fit for ARTickle
Capacitor (recommended)	Yes	Yes	Best — reuses the existing Vite dist build; one codebase, both stores
PWA → TWA (Bubblewrap / PWABuilder)	Yes	Weak	Android-mainly; needs a real service worker; iOS support is poor

Recommendation: use **Capacitor**. It wraps the current build with minimal changes and ships to both stores.

1. App-readiness fixes specific to our code

These will cause App Store rejection or broken behaviour inside a native shell. Do them before wrapping.

- **Stop loading code from CDNs.** The app currently pulls React, Tailwind, jsPDF, SheetJS and JSZip from CDNs at runtime. Apple guideline 2.5.2 restricts apps that download and execute remote code, and it breaks offline launch. **Bundle these via npm + Vite instead.** This is the single biggest blocker.
- **Add “Sign in with Apple”.** The app already offers **Google login**, and Apple **requires** Sign in with Apple on iPhone whenever any third-party social login is present (Apple guideline 4.8). Without it the app is rejected. Only appears on the iPhone version — Android and the website are unaffected. Depends on the Apple Developer account being active.
- **No service worker yet** — the app is not a true installable PWA. Required for the TWA route; recommended for Capacitor offline reliability.
- ✓ **Already in place:** web manifest, app icons and theme-color meta tags. Icon and splash sizes will need to be expanded for native.

2. Accounts & tooling

- **Apple Developer Program** — USD \$99 / year. Requires a Mac with Xcode for iOS builds.
- **Google Play Developer account** — USD \$25 one-time.
- Xcode + iOS simulator/device; Android Studio + emulator for local builds and testing.

3. Legal & compliance (both stores enforce these)

- **Privacy policy URL** — publicly hosted (can live on Firebase Hosting).
- **In-app account deletion** — because the app has user accounts, Apple (5.1.1 v) and Google both require a way for users to delete their account **from inside the app**, not just by email.
- **Apple Privacy “Nutrition Label”** and **Google Play Data Safety form** — declare what Firestore / Cloud Functions collect.
- **Content / age rating** questionnaire on both stores.

- ■ **Sign in with Apple** — **mandatory for us**, because the app already offers Google login (see Section 1). Built during the coding phase.
- **App Tracking Transparency** prompt on iOS only if we do cross-app tracking (likely not applicable).

4. Store listing assets (per store)

- App name, subtitle, full description, keywords.
- **Icon** — 1024×1024 (Apple), 512×512 (Google Play).
- **Screenshots** for each required device size (iPhone 6.7" / 6.5", iPad if supported; Android phone/tablet).
- **Feature graphic** 1024×500 (Google Play).
- Support URL, app category, and privacy policy URL.

5. Build, sign & submit

- Set bundle IDs (e.g. `com.artickle.academy`), version and build number.
- **iOS**: signing certificates + provisioning profiles → Archive in Xcode → upload to App Store Connect → test via TestFlight first.
- **Android**: generate a signing key → build an **AAB** (not APK) → upload to Play Console → use the internal testing track first.
- Submit for review. Typical timelines: Apple ~24–48h; Google hours to a few days.

6. What it costs

Item	Cost	Frequency	Paid by
Claude Max 5× plan (to do the build work)	\$100	per month	You
Apple Developer Program	\$99	per year	You
Google Play Developer account	\$25	one-time	You

Realistic first-launch outlay ≈ \$224 (\$100 Max + \$99 Apple + \$25 Google). The whole build should fit inside **one month** of the Max plan. On a subscription you are **not billed per token** — you pay the flat monthly fee and simply have usage limits.

Which Claude model: use **Opus 4.8** (the most capable) for the build — since you are not coding, reliability matters more than saving on usage. The Max 5× plan comfortably covers a project of this size.

7. Who does what, and how long

Task	Who	Time
Bundle the CDN code locally (biggest blocker)	Claude	~½ day
Wrap the app with Capacitor (iOS + Android)	Claude	~1 day
Add in-app account deletion	Claude	~½ day
Add Sign in with Apple (iPhone)	Claude	~½–1 day
Generate icons & splash screens from your logo	Claude	~1–2 hrs
Register Apple & Google developer accounts	You	~1 hr setup*
Apple “Services ID” + key (for Sign in with Apple)	You (guided)	~30 min
Privacy policy text	You (Claude drafts)	~1 hr
Screenshots & store description	You + Claude	a few hrs
Final submission (needs your logins + a Mac)	You (guided)	~½ day

* Apple account approval itself can take 1–2 days — worth starting early.

Coding effort: ~3.5–5 working days of Claude’s work. **Calendar time to live in both stores:** typically **2–4 weeks**, mostly waiting on Apple account approval and store review.

8. Updating the app after launch

- **✓ Quick fixes & content changes** (text, prices, small logic) can be pushed **instantly** — **no waiting for approval** — if we set up a “live update” mechanism during the build. Allowed by both stores for bug fixes and content. This keeps the fast workflow you have today.
- **■ New features or bigger changes** need a **fresh submission + review**: Apple usually ~1 day, Google hours to a day. You don’t rebuild anything — you resubmit and wait for approval.
- Bottom line: it is **not** “wait for approval every single time.” Most day-to-day tweaks stay instant; only meaningful new features go through review.

Critical path: (1) bundle the CDN dependencies locally → (2) add Capacitor → (3) add account deletion + Sign in with Apple + privacy policy → (4) generate store assets → (5) submit.

Legend: ■ = blocker / action required ✓ = already in place.